

CONTRAT D'API REST

EduGest CM — Gestion École Primaire Bilingue — Cameroun

Version	1.0.0 — Alignée sur le MCD/MPD enseignant
Backend	Node.js + Express.js
ORM	Sequelize — dialect MySQL
Base de données	MySQL Schéma : edugest_db
Auth	JWT Bearer Token Expiration : 24h
Base URL	http://localhost:8080/api
Frontend	React 18
Rôles RBAC	ADMIN DIRECTEUR ENSEIGNANT SECRETAIRE PARENT
Tables MPD	27 tables imposées — aucune modification de schéma
Équipe	ENSP 3GI — Projet 2024-2025

Ce contrat est strictement basé sur le MPD fourni par l'enseignant. Chaque endpoint indique les tables MySQL exactes utilisées. Aucune table supplémentaire n'est créée.

TABLE DES MATIÈRES

1. Conventions générales & Authentification → [Personne](#), [Admin](#)
2. Module Années Académiques & Trimestres → [AnneeAcademique](#), [Trimestre](#), [Session](#)
3. Module Cycles & Classes → [Cycle](#), [Classe](#), [Salle](#), [Titulaire](#)
4. Module Élèves → [Eleve](#), [VilleNaissance](#), [Parents](#), [Residents](#), [Quartier](#)
5. Module Enseignants → [Enseignant](#), [Personne](#), [FicheEnseignant](#)
6. Module Matières (Cours) → [Cours](#), [Specialite](#), [Livres](#), [NatureEpreuve](#)
7. Module Emploi du Temps → [EmploiDuTemps](#)
8. Module Présences → [Frequente](#)
9. Module Évaluations & Notes → [Epreuve](#), [Evaluation](#), [Session](#)
10. Module Bulletins (calculés) → [Evaluation](#), [Cours](#), [Session](#) — [calcul Node.js](#)
11. Module Scolarité & Paiements → [Scolarite](#), [Tranches](#), [Paiement](#), [Mode](#)
12. Module Discipline → [Rapport](#), [Discipline](#), [Justificatifs](#)
13. Module Communication → [Messages](#), [Parents](#), [Personne](#)
14. Module Rapports & Exports → [Toutes tables](#) — [génération PDF dynamique](#)
15. Codes HTTP standardisés
16. Correspondance MPD → API → [Tableau complet](#)

1. CONVENTIONS GÉNÉRALES & AUTHENTIFICATION

Architecture de la requête

Toutes les requêtes protégées doivent inclure :

Authorization: Bearer

Content-Type: application/json

Base URL développement

http://localhost:8080/api

Paramètres de pagination (GET liste)

?page=0&size;=20&sort;=nom=ASC

Structure JWT — Payload décodé

```
{
  "idPers" : 1, // ID dans table Personne ou Admin
  "username" : "mbarga.dir",
  "role" : "DIRECTEUR", // ADMIN|DIRECTEUR|ENSEIGNANT|SECRETAIRE|PARENT
  "typePersonne": 2, // champ typePersonne de la table Personne
  "iat" : 1700000000,
  "exp" : 1700086400 // iat + 24h
}
```

Mapping rôles ↔ table Personne.typePersonne

Rôle JWT	typePersonne	Source table	Accès
ADMIN	—	Admin	Toutes routes
DIRECTEUR	2	Personne	Gestion complète
ENSEIGNANT	1	Personne	Ses classes/cours
SECRETAIRE	3	Personne	Élèves, paiements
PARENT	4	Personne	Son enfant uniquement

Réponse d'erreur standardisée

```
{
  "timestamp": "2024-11-18T10:30:00Z",
  "status" : 403,
  "error" : "Forbidden",
  "message" : "Acces refuse : role ENSEIGNANT insuffisant",
  "path" : "/api/eleves"
}
```

Endpoints d'authentification

Tables BD : [Personne](#) | [Admin](#)

Méthode	Endpoint	Rôles	Description	Tables BD
POST	/auth/login	PUBLIC	Connexion — retourne JWT	Personne , Admin
POST	/auth/logout	Tous (JWT)	Invalidation du token	—
GET	/auth/me	Tous (JWT)	Profil utilisateur connecté	Personne
PATCH	/auth/password	Tous (JWT)	Changer son mot de passe	Personne

POST /auth/login — Connexion

Rôles : [PUBLIC]

Tables MySQL : **Personne Admin**

Corps de la requête (JSON) :

```
// Node.js vérifie d'abord la table Admin, puis Personne
{
  "username" : "mbarga.directeur", // champ username (Personne ou Admin)
  "password" : "motdepasse123" // comparé via bcrypt
}
```

Réponse (JSON) :

```
{
  "token" : "eyJhbGciOiJIUzI1NiJ9...",
  "expiresIn" : 86400,
  "utilisateur": {
    "idPers" : 1,
    "nom" : "MBARGA",
    "prenom" : "Jean",
    "role" : "DIRECTEUR",
    "typePersonne" : 2
  }
}
```

Code	Signification
200	Connexion réussie — token retourné
401	Username ou mot de passe incorrect
423	Compte inactif (actif = 0)

2. MODULE ANNÉES ACADÉMIQUES & TRIMESTRES

Tables BD : [AnneeAcademique](#) | [Trimestre](#) | [Session](#)

Ces trois tables structurent le calendrier scolaire. [AnneeAcademique](#) contient l'année (ex : 2024-2025). [Trimestre](#) la divise en 3 périodes. [Session](#) subdivise chaque trimestre en séquences d'évaluation.

Méthode	Endpoint	Rôles	Description	Tables BD
GET	/annees	ADMIN, DIRECTEUR	Lister les années académiques	AnneeAcademique
POST	/annees	ADMIN	Créer une année académique	AnneeAcademique
GET	/annees/courante	Tous (JWT)	Année académique en cours	AnneeAcademique
GET	/trimestres	Tous (JWT)	Lister les trimestres	Trimestre
POST	/trimestres	ADMIN, DIRECTEUR	Créer un trimestre	Trimestre
GET	/trimestres/:id	Tous (JWT)	Détail d'un trimestre	Trimestre
GET	/sessions	Tous (JWT)	Lister les sessions	Session
POST	/sessions	ADMIN, DIRECTEUR	Créer une session	Session
GET	/sessions/:id	Tous (JWT)	Détail d'une session	Session

POST /annees — Créer une année académique

Rôles : [\[ADMIN\]](#)

Tables MySQL : [AnneeAcademique](#)

Corps de la requête (JSON) :

```
// Table AnneeAcademique : idAnnee, libelle, periode, created_at, idAdmin
{
  "libelle" : "Annee Scolaire 2024-2025",
  "periode" : "Septembre 2024 - Juin 2025"
}
```

Réponse (JSON) :

```
{
  "idAnnee" : 1,
  "libelle" : "Annee Scolaire 2024-2025",
  "periode" : "Septembre 2024 - Juin 2025",
  "message" : "Annee academique creee"
}
```

Code	Signification
201	Créée
400	Données invalides
409	Année déjà existante

POST /sessions — Créer une session (séquence d'évaluation)

Rôles : [\[ADMIN\]](#) [\[DIRECTEUR\]](#)

Tables MySQL : [Session Trimestre](#)

Corps de la requête (JSON) :

```
// Table Session : idSession, libelle, description, idTrimestre, idPers
{
  "libelle" : "Sequence 1",
  "description" : "Premiere sequence du trimestre 1",
  "idTrimestre" : 1
}
```

Réponse (JSON) :

```
{
  "idSession" : 1,
  "libelle" : "Sequence 1",
  "idTrimestre": 1,
  "message" : "Session creee"
}
```

Code	Signification
201	Créée
404	Trimestre introuvable

3. MODULE CYCLES & CLASSES

Tables BD : [Cycle](#) | [Classe](#) | [Salle](#) | [Titulaire](#)

Cycle = niveau pédagogique (CP, CE, CM). Classe = une classe concrète liée à un Cycle. Salle = salle physique liée à une Classe. Titulaire = enseignant titulaire d'une Salle.

Méthode	Endpoint	Rôles	Description	Tables BD
GET	/cycles	Tous (JWT)	Lister les cycles	Cycle
POST	/cycles	ADMIN	Créer un cycle	Cycle
GET	/classes	ADMIN, DIRECTEUR, ENSEIGNANT	Lister les classes	Classe , Cycle
POST	/classes	ADMIN, DIRECTEUR	Créer une classe	Classe
GET	/classes/:id	ADMIN, DIRECTEUR, ENSEIGNANT	Détail d'une classe	Classe , Salle , Titulaire
PUT	/classes/:id	ADMIN, DIRECTEUR	Modifier une classe	Classe
DELETE	/classes/:id	ADMIN	Supprimer une classe	Classe
GET	/classes/:id/eleves	ADMIN, DIRECTEUR, ENSEIGNANT	Élèves d'une classe	Eleve , Frequence
GET	/classes/:id/cours	Tous (JWT)	Cours affectés à la classe	EmploiDuTemps , Cours
GET	/salles	ADMIN, DIRECTEUR	Lister les salles	Salle
POST	/salles	ADMIN, DIRECTEUR	Créer une salle	Salle
POST	/salles/:id/titulaire	ADMIN, DIRECTEUR	Affecter enseignant titulaire	Titulaire , Personne

POST /classes — Créer une classe

Rôles : [\[ADMIN\]](#) [\[DIRECTEUR\]](#)

Tables MySQL : [Classe](#) [Cycle](#)

Corps de la requête (JSON) :

```
// Table Classe : idClasse, libelle, idCycle, idAdmin, created_at
{
  "libelle" : "CM2-A",
  "idCycle" : 3
}
```

Réponse (JSON) :

```
{
  "idClasse": 6,
  "libelle" : "CM2-A",
  "cycle" : { "idCycle": 3, "libelle": "Cours Moyen" },
  "message" : "Classe creee"
}
```

Code	Signification
201	Créée
404	Cycle introuvable
409	Classe déjà existante

GET /classes/:id — Détail d'une classe avec salle et titulaire

Rôles : [\[ADMIN\]](#) [\[DIRECTEUR\]](#) [\[ENSEIGNANT\]](#)

Tables MySQL : [Classe](#) [Salle](#) [Titulaire](#) [Personne](#)

Réponse (JSON) :

```
{
  "idClasse" : 1,
  "libelle" : "CM2-A",
  "cycle" : { "idCycle": 3, "libelle": "Cours Moyen" },
  "salle" : {
    "idSalle" : 1, "libelle": "Salle 1",
    "position": "Bat A", "surface": "60m2"
  },
  "titulaire" : {
    "idPers" : 2, "nom": "FOUDA", "prenom": "Marie"
  },
  "effectif" : 32
}
```

Code	Signification
200	Retourné
404	Classe introuvable

4. MODULE ÉLÈVES

Tables BD : [Eleve](#) | [VilleNaissance](#) | [Parents](#) | [Personne](#) | [Residents](#) | [Quartier](#)

La table Eleve utilise matricule (INT) comme clé primaire — généré automatiquement par Node.js. Le champ langue stocke la section (FR/EN/BIL). La table Parents lie un Eleve à une Personne (tuteur légal).

Méthode	Endpoint	Rôles	Description	Tables BD
GET	/eleves	ADMIN, DIRECTEUR, SECRETAIRE	Liste paginée des élèves	Eleve , VilleNaissance
POST	/eleves	ADMIN, DIRECTEUR, SECRETAIRE	Inscrire un nouvel élève	Eleve
GET	/eleves/:matricule	ADMIN, DIRECTEUR, SECRETAIRE, PARENT*	Dossier complet	Eleve , VilleNaissance , Parents
PUT	/eleves/:matricule	ADMIN, DIRECTEUR, SECRETAIRE	Modifier dossier	Eleve
DELETE	/eleves/:matricule	ADMIN, DIRECTEUR	Archiver (actif = 0)	Eleve
GET	/eleves/:matricule/notes	ADMIN, DIRECTEUR, ENSEIGNANT, PARENT*	Notes de l'élève	Evaluation , Cours , Session
GET	/eleves/:matricule/presences	ADMIN, DIRECTEUR, ENSEIGNANT, PARENT*	Présences élève	Frequent , Salle , AnneeAcademique
GET	/eleves/:matricule/paiements	ADMIN, DIRECTEUR, SECRETAIRE, PARENT*	Paiements élève	Paiement , Mode
GET	/eleves/:matricule/bulletin	ADMIN, DIRECTEUR, ENSEIGNANT, PARENT*	Bulletin calculé	Evaluation , Cours , Session , Trimestre
GET	/eleves/:matricule/discipline	ADMIN, DIRECTEUR	Historique disciplinaire	Rapport , Discipline
GET	/eleves/:matricule/parents	ADMIN, DIRECTEUR, SECRETAIRE	Tuteurs légaux	Parents , Personne
POST	/eleves/:matricule/parents	ADMIN, DIRECTEUR, SECRETAIRE	Associer un tuteur	Parents , Personne

* PARENT : vérifié que Parents.matricule correspond au matricule demandé.

POST /eleves — Inscrire un nouvel élève

Rôles : [ADMIN] [DIRECTEUR] [SECRETAIRE]

Tables MySQL : Eleve VilleNaissance

■ Le matricule est généré par Node.js : ANNEE + séquence auto (ex: 2024001). Le champ langue stocke la section FR/EN/BIL.
idAdmin = idPers du connecté.

Corps de la requête (JSON) :

```
// Champs de la table Eleve
{
  "nom" : "OWONA",
  "prenom" : "Jules",
  "dateNaissance" : "2018-07-22",
  "lieuNaissance" : "Yaounde",
  "sexe" : 1, // 0=fille, 1=garcon, 2=autres
  "langue" : "FR", // FR | EN | BIL (section scolaire)
  "idVilleNaissance" : 1,
  "photoURL" : "" // optionnel
}
```

Réponse (JSON) :

```
{
  "matricule" : 2024007, // genere automatiquement
  "nom" : "OWONA",
  "prenom" : "Jules",
  "actif" : 1,
  "message" : "Eleve inscrit avec succes"
}
```

Code	Signification
201	Inscrit
400	Données invalides
409	Matricule ou nom/date déjà existant

GET /eleves/:matricule — Dossier complet d'un élève

Rôles : [ADMIN] [DIRECTEUR] [SECRETAIRE] [PARENT]

Tables MySQL : Eleve VilleNaissance Parents Personne

Réponse (JSON) :

```
{
  "matricule" : 2024001,
  "nom" : "ATANGA",
  "prenom" : "Lionel",
  "dateNaissance" : "2016-03-15",
  "lieuNaissance" : "Yaounde",
  "sexe" : 1,
  "langue" : "FR",
  "actif" : 1,
  "villeNaissance" : { "idVille": 1, "libelle": "Yaounde" },
  "parents" : [
    {
      "idParent" : 1,
      "lienParente": "PERE",
      "personne" : { "idPers": 10, "nom": "ATANGA", "prenom": "Paul",
        "mobile": "+237677123456" }
    }
  ],
  "adresse" : {
    "quartier" : "Biyem-Assi",
    "description": "Rue des ecoles"
  }
}
```

Code	Signification
200	Retourné
403	Parent accédant à un autre élève
404	Élève introuvable

5. MODULE ENSEIGNANTS

Tables BD : [Enseignant](#) | [Personne](#) | [FicheEnseignant](#) | [Titulaire](#)

Un Enseignant est lié à une Personne (idPers) et à un Cours (idCours). FicheEnseignant stocke les rapports/incidents concernant un enseignant. Titulaire lie un enseignant à une Salle comme responsable de classe.

Méthode	Endpoint	Rôles	Description	Tables BD
GET	/enseignants	ADMIN, DIRECTEUR	Lister les enseignants	Enseignant, Personne
GET	/enseignants/:id	ADMIN, DIRECTEUR	Détail enseignant	Enseignant, Personne, Cours
POST	/enseignants	ADMIN, DIRECTEUR	Créer profil enseignant	Enseignant, Personne
PUT	/enseignants/:id	ADMIN, DIRECTEUR	Modifier profil	Enseignant, Personne
DELETE	/enseignants/:id	ADMIN	Désactiver (Actif = 0)	Enseignant
GET	/enseignants/:id/cours	ADMIN, DIRECTEUR	Cours enseignés	Enseignant, Cours
GET	/enseignants/:id/classes	ADMIN, DIRECTEUR	Classes affectées	Titulaire, Salle, Classe
GET	/enseignants/:id/fiches	ADMIN, DIRECTEUR	Fiches / rapports enseignant	FicheEnseignant
POST	/enseignants/:id/fiches	ADMIN, DIRECTEUR	Créer une fiche enseignant	FicheEnseignant

POST /enseignants — Créer un profil enseignant

Rôles : [\[ADMIN\]](#) [\[DIRECTEUR\]](#)

Tables MySQL : [Personne Enseignant](#)

■ Crée d'abord une Personne (typePersonne=1) puis un Enseignant lié.

Corps de la requête (JSON) :

```
// Étape 1 – champs Personne
{
  "nom" : "NKOLO",
  "prenom" : "Brigitte",
  "dateNaissance": "1985-04-10",
  "lieuNaissance": "Bafoussam",
  "mobile" : "+237677001122",
  "phone" : "+237699001122",
  "username" : "nkolo.brigitte",
  "password" : "motdepasse", // bcrypt en base
  "typePersonne" : 1, // 1 = Enseignant
// Étape 2 – champs Enseignant
  "idCours" : 1, // matiere principale
  "actif" : 1
}
```

Réponse (JSON) :

```
{
  "idEnseignant" : 5,
  "idPers" : 12,
  "nom" : "NKOLO",
  "prenom" : "Brigitte",
  "idCours" : 1,
  "message" : "Profil enseignant cree"
}
```

Code	Signification
201	Créé
409	Username déjà utilisé
400	Données invalides

6. MODULE MATIÈRES (COURS)

Tables BD : [Cours](#) | [Specialite](#) | [Livres](#) | [NatureEpreuve](#)

La table Cours représente les matières enseignées. Elle est liée à Livres (manuel scolaire). Specialite catégorise les livres. NatureEpreuve définit les types d'épreuves (Contrôle Continu, Composition, Devoir, Examen).

Méthode	Endpoint	Rôles	Description	Tables BD
GET	/cours	Tous (JWT)	Lister toutes les matières	Cours, Livres
GET	/cours/:id	Tous (JWT)	Détail d'une matière	Cours, Livres, Specialite
POST	/cours	ADMIN	Créer une matière	Cours
PUT	/cours/:id	ADMIN	Modifier une matière	Cours
GET	/specialites	Tous (JWT)	Lister les spécialités	Specialite
POST	/specialites	ADMIN	Créer spécialité	Specialite
GET	/livres	ADMIN, DIRECTEUR	Catalogue des livres	Livres, Specialite
POST	/livres	ADMIN	Ajouter un livre	Livres
GET	/nature-epreuves	Tous (JWT)	Types d'épreuves disponibles	NatureEpreuve
POST	/nature-epreuves	ADMIN	Créer un type d'épreuve	NatureEpreuve

POST /cours — Créer une matière

Rôles : [\[ADMIN\]](#)

Tables MySQL : [Cours](#)

Corps de la requête (JSON) :

```
// Table Cours : idCours, libelle, note, coefficient, description, idLivre, actif, idAdmin
{
  "libelle" : "Langue Francaise",
  "coefficient" : 3.0,
  "description" : "Lecture, expression ecrite, grammaire",
  "idLivre" : 1, // manuel associe (optionnel)
  "actif" : 1
}
// Note : le champ 'note' de la table Cours represente la note maximale (default 20)
```

Réponse (JSON) :

```
{
  "idCours" : 1,
  "libelle" : "Langue Francaise",
  "coefficient" : 3.0,
  "actif" : 1,
  "message" : "Matiere creee"
}
```

Code	Signification
201	Créée
409	Matière déjà existante

7. MODULE EMPLOI DU TEMPS

Tables BD : [EmploiDuTemps](#) | [Classe](#) | [Cours](#) | [Enseignant](#)

La table EmploiDuTemps stocke les créneaux horaires. Champs disponibles : jour (varchar), heure (varchar 6 car.), idClasse, idCours, idAdmin. Le champ heureFin n'existe pas en base — le front calcule la durée depuis heure.

Méthode	Endpoint	Rôles	Description	Tables BD
GET	/emploiutemps	ADMIN, DIRECTEUR, ENSEIGNANT	Tous les créneaux	EmploiDuTemps
GET	/emploiutemps/classe/:id	ADMIN, DIRECTEUR, ENSEIGNANT	EDT d'une classe	EmploiDuTemps , Cours
GET	/emploiutemps/cours/:id	ADMIN, DIRECTEUR, ENSEIGNANT	Créneaux d'une matière	EmploiDuTemps , Classe
POST	/emploiutemps	ADMIN, DIRECTEUR	Ajouter un créneau	EmploiDuTemps
PUT	/emploiutemps/:id	ADMIN, DIRECTEUR	Modifier un créneau	EmploiDuTemps
DELETE	/emploiutemps/:id	ADMIN, DIRECTEUR	Supprimer un créneau	EmploiDuTemps

POST /emploiutemps — Ajouter un créneau à l'emploi du temps

Rôles : [\[ADMIN\]](#) [\[DIRECTEUR\]](#)

Tables MySQL : [EmploiDuTemps](#)

Corps de la requête (JSON) :

```
// Table EmploiDuTemps : idTemps, jour, heure, idClasse, idCours, idAdmin
{
  "jour" : "LUNDI", // LUNDI|MARDI|MERCREDI|JEUDI|VENDREDI|SAMEDI
  "heure" : "07:30", // heure de debut (format HH:MM, varchar 6)
  "idClasse": 1,
  "idCours" : 1
}
```

Réponse (JSON) :

```
{
  "idTemps" : 1,
  "jour" : "LUNDI",
  "heure" : "07:30",
  "classe" : { "idClasse": 1, "libelle": "CM2-A" },
  "cours" : { "idCours": 1, "libelle": "Langue Francaise", "coefficient": 3 },
  "message" : "Creneau ajoute"
}
```

Code	Signification
201	Ajouté
409	Conflit : créneau déjà existant pour cette classe ce jour/heure

8. MODULE PRÉSENCES (Frequente)

Tables BD : **Frequente** | **Eleve** | **Salle** | **AnneeAcademique**

La table Frequente gère l'appel quotidien. Workaround imposé par le MPD : le champ commentaire (varchar 255) stocke le statut de présence sous la forme 'STATUT|motif' ex: 'ABSENT|Maladie' ou 'PRESENT|RAS' ou 'RETARD|Transport'. Node.js parse ce champ automatiquement à la lecture.

Méthode	Endpoint	Rôles	Description	Tables BD
GET	/presences	ADMIN, DIRECTEUR	Toutes les présences (filtrable)	Frequent e
GET	/presences/classe/:id	ADMIN, DIRECTEUR, ENSEIGNANT	Présences d'une classe pour une date	Frequent e, Eleve
POST	/presences/appel	ENSEIGNANT	Saisir l'appel complet d'une classe	Frequent e
PUT	/presences/:id	ENSEIGNANT, DIRECTEUR	Modifier un statut de présence	Frequent e
GET	/presences/stats	ADMIN, DIRECTEUR	Statistiques de présences	Frequent e
GET	/eleves/:matricule/presences	ADMIN, DIRECTEUR, ENSEIGNANT, PARENT*	Présences d'un élève	Frequent e

POST /presences/appel — Saisir l'appel complet d'une classe

Rôles : **[ENSEIGNANT]**

Tables MySQL : **Frequente** **Eleve** **Salle** **AnneeAcademique**

■ Workaround MPD : commentaire = 'STATUT|motif'. Node.js insère une ligne Frequente par élève. 409 si une ligne Frequente existe déjà pour ce (matricule, idSalle, idAcademi) ce jour.

Corps de la requête (JSON) :

```
// Une requête = tout l'appel de la classe
{
  "idSalle" : 1, // salle de la classe
  "idAcademi" : 1, // AnneeAcademique en cours
  "date" : "2024-11-18",
  "presences" : [
    { "matricule": 2024001, "statut": "PRESENT", "motif": "" },
    { "matricule": 2024002, "statut": "ABSENT", "motif": "Maladie" },
    { "matricule": 2024003, "statut": "RETARD", "motif": "Transport" }
  ]
}
// Node.js stocke : commentaire = "PRESENT|" ou "ABSENT|Maladie" ou "RETARD|Transport"
```

Réponse (JSON) :

```
{
  "message" : "Appel enregistré",
  "date" : "2024-11-18",
  "idSalle" : 1,
  "totalEleves" : 32,
  "presents" : 29,
  "absents" : 2,
  "retards" : 1,
  "tauxPresence": "90.6%"
}
```

Code	Signification
201	Appel enregistré
409	Appel déjà saisi pour cette salle/date
403	Enseignant non titulaire de cette salle

9. MODULE ÉVALUATIONS & NOTES

Tables BD : [Epreuve](#) | [Evaluation](#) | [Session](#) | [Cours](#) | [NatureEpreuve](#)

Architecture en 2 niveaux imposée par le MPD : Epreuve = définition du devoir/examen. Evaluation = note individuelle d'un élève (matricule + note + idEpreuve + idCours + idSession). Une Epreuve peut avoir plusieurs Evaluations (une par élève).

Méthode	Endpoint	Rôles	Description	Tables BD
GET	/epreuves	ADMIN, DIRECTEUR, ENSEIGNANT	Lister les épreuves	Epreuve, NatureEpreuve
POST	/epreuves	ENSEIGNANT, DIRECTEUR	Créer une épreuve	Epreuve
GET	/epreuves/:id	ADMIN, DIRECTEUR, ENSEIGNANT	Détail épreuve	Epreuve, Evaluation
POST	/epreuves/:id/notes	ENSEIGNANT	Saisir les notes d'une épreuve	Evaluation
GET	/evaluations	ADMIN, DIRECTEUR, ENSEIGNANT	Lister les évaluations	Evaluation, Epreuve, Cours
GET	/evaluations/:id	ADMIN, DIRECTEUR, ENSEIGNANT	Détail d'une évaluation	Evaluation, Cours, Epreuve
PUT	/evaluations/:id	ENSEIGNANT, DIRECTEUR	Modifier une note	Evaluation
GET	/eleves/:matricule/notes	ADMIN, DIRECTEUR, ENSEIGNANT, PARENT*	Toutes les notes d'un élève	Evaluation, Cours, Session
GET	/classes/:id/moyennes	ADMIN, DIRECTEUR, ENSEIGNANT	Moyennes d'une classe	Evaluation, Cours
GET	/classes/:id/classement	ADMIN, DIRECTEUR, ENSEIGNANT	Classement d'une classe	Evaluation, Cours
GET	/nature-epreuves	Tous (JWT)	Types d'épreuves	NatureEpreuve

POST /epreuves — Créer une épreuve (devoir, composition...)

Rôles : [ENSEIGNANT] [DIRECTEUR]

Tables MySQL : Epreuve NatureEpreuve

Corps de la requête (JSON) :

```
// Table Epreuve : idEpreuve, libelle, urlDoc, auteur, idNature, idPers, created_at
{
  "libelle" : "Composition de Francais T1",
  "urlDoc" : "", // lien vers sujet (optionnel)
  "auteur" : "FOUDA Marie",
  "idNature" : 1 // NatureEpreuve : Controle|Composition|Devoir|Examen
}
```

Réponse (JSON) :

```
{
  "idEpreuve": 8,
  "libelle" : "Composition de Francais T1",
  "nature" : { "idNature": 1, "libelle": "Composition" },
  "message" : "Epreuve creee"
}
```

Code	Signification
201	Créée
403	Non autorisé

POST /epreuves/:id/notes — Saisir les notes de tous les élèves

Rôles : [ENSEIGNANT]

Tables MySQL : Evaluation Epreuve Cours Session Eleve

■ Crée une ligne Evaluation par élève. Evaluation.note = note de l'élève. Evaluation.idEpreuve lie à l'épreuve. Evaluation.idCours et idSession doivent être fournis.

Corps de la requête (JSON) :

```
// Table Evaluation : idEval, note, appreciation, matricule, idEpreuve, idCours, idSession, idPers
{
  "idCours" : 1,
  "idSession": 1,
  "notes" : [
    { "matricule": 2024001, "note": 14.5, "appreciation": "Bien" },
    { "matricule": 2024002, "note": 16.0, "appreciation": "Tres bien" },
    { "matricule": 2024003, "note": null, "appreciation": "Absent" }
  ]
}
```

Réponse (JSON) :

```
{
  "message" : "Notes enregistrees",
  "idEpreuve" : 8,
  "notesSaisies": 32,
  "moyenne" : 13.3,
  "noteMin" : 8.0,
  "noteMax" : 18.5
}
```

Code	Signification
201	Notes enregistrées
400	Note hors plage
409	Notes déjà saisies pour cette épreuve

10. MODULE BULLETINS (calculés par Node.js)

Tables BD : [Evaluation](#) | [Cours](#) | [Session](#) | [Trimestre](#) | [Eleve](#)

Il n'existe pas de table Bulletin dans le MPD de l'enseignant. Node.js calcule le bulletin à la volée en agrégeant les Evaluations d'un élève pour un trimestre donné, pondérées par les coefficients des Cours.

Formule de calcul Node.js :

```
// Algorithme bulletin (bulletinService.js)
const evals = await Evaluation.findAll({
  where : { matricule },
  include : [{ model: Cours }, { model: Session, include: [Trimestre] }]
});
const parTrimestre = evals.filter(e => e.Session.Trimestre.idTrimes == idTrimes);

let totalPoints = 0, totalCoeff = 0;
parTrimestre.forEach(e => {
  totalPoints += e.note * e.Cours.coefficient;
  totalCoeff += e.Cours.coefficient;
});
const moyenneGenerale = totalCoeff > 0 ? (totalPoints / totalCoeff).toFixed(2) : 0;
```

Méthode	Endpoint	Rôles	Description	Tables BD
GET	/bulletins/eleve/:matricule	ADMIN, DIRECTEUR, ENSEIGNANT, PARENT*	Bulletin calculé d'un élève	Evaluation, Cours, Session, Trimestre
GET	/bulletins/classe/:id	ADMIN, DIRECTEUR, ENSEIGNANT	Bulletins de toute une classe	Evaluation, Cours, Eleve
GET	/bulletins/eleve/:matricule/pdf	ADMIN, DIRECTEUR, PARENT*	Télécharger bulletin PDF	Evaluation, Cours, Eleve

GET /bulletins/eleve/:matricule — Bulletin scolaire calculé d'un élève

Rôles : [\[ADMIN\]](#) [\[DIRECTEUR\]](#) [\[ENSEIGNANT\]](#) [\[PARENT\]](#)

Tables MySQL : [Evaluation Cours Session Trimestre Eleve](#)

Corps de la requête (JSON) :

```
// ?idTrimes=1 (paramètre de filtre – idTrimes de la table Trimestre)
```

Réponse (JSON) :

```
{
  "eleve" : { "matricule": 2024001, "nom": "ATANGA", "prenom": "Lionel" },
  "trimestre": { "idTrimes": 1, "libelle": "Trimestre 1" },
  "notes" : [
    {
      "idCours" : 1,
      "matiere" : "Langue Francaise",
      "coefficient" : 3,
      "note" : 14.5,
      "moyPonderee" : 43.5,
      "appreciation": "Bien"
    },
    {
      "idCours" : 2,
      "matiere" : "Mathematiques",
      "coefficient" : 3,
      "note" : 15.0,
      "moyPonderee" : 45.0,
      "appreciation": "Bien"
    }
  ],
  "moyenneGenerale": 14.75,
  "rang" : 3,
  "totalEleves" : 29,
  "mention" : "BIEN"
}
```

Code	Signification
200	Bulletin calculé et retourné
400	Notes incomplètes pour ce trimestre
404	Élève introuvable

11. MODULE SCOLARITÉ & PAIEMENTS

Tables BD : [Scolarite](#) | [Tranches](#) | [Paielement](#) | [Mode](#)

Scolarite définit les montants par Cycle (inscription + pension). Tranches décompose la pension en échéances. Paiement enregistre chaque versement d'un élève. Mode liste les modes de paiement (Espèces, Mobile Money, Chèque, Virement).

Workaround : le champ commentaire de Paiement stocke le type (SCOLARITE/CANTINE/APE/AUTRE).

Méthode	Endpoint	Rôles	Description	Tables BD
GET	/scolarites	ADMIN, DIRECTEUR	Grilles tarifaires par cycle	Scolarite , Cycle
POST	/scolarites	ADMIN	Créer grille tarifaire	Scolarite
GET	/scolarites/:id/tranches	ADMIN, DIRECTEUR, SECRETAIRE	Tranches d'une scolarité	Tranches
POST	/scolarites/:id/tranches	ADMIN	Ajouter une tranche	Tranches
GET	/modes	Tous (JWT)	Modes de paiement disponibles	Mode
GET	/paiements	ADMIN, DIRECTEUR, SECRETAIRE	Tous les paiements	Paielement , Mode , Eleve
GET	/paiements/:id	ADMIN, DIRECTEUR, SECRETAIRE	Détail d'un paiement	Paielement , Mode
POST	/paiements	SECRETAIRE, DIRECTEUR	Enregistrer un paiement	Paielement
PUT	/paiements/:id	SECRETAIRE, DIRECTEUR	Modifier paiement	Paielement
DELETE	/paiements/:id	ADMIN, DIRECTEUR	Annuler un paiement	Paielement
GET	/paiements/impayes	ADMIN, DIRECTEUR, SECRETAIRE	Élèves avec impayés	Paielement , Eleve , Scolarite
GET	/paiements/stats	ADMIN, DIRECTEUR	Statistiques financières	Paielement , AnneeAcademique
GET	/eleves/:matricule/paiements	ADMIN, DIRECTEUR, SECRETAIRE, PARENT*	Paiements d'un élève	Paielement , Mode

POST /paiements — Enregistrer un paiement

Rôles : [SECRETAIRE] [DIRECTEUR]

Tables MySQL : Paiement Mode Eleve AnneeAcademique

■ Workaround MPD : le champ commentaire stocke le type de frais sous la forme 'TYPE|trimestre' ex: 'SCOLARITE|T1' ou 'APE|' ou 'CANTINE|T2'.

Corps de la requête (JSON) :

```
// Table Paiement : idPaie, matricule, idAca, montant, url,
// commentaire, idMode, operation_ID, idPers, datePaie, dateEnregistrer
{
  "matricule" : 2024001,
  "idAca" : 1, // AnneeAcademique
  "montant" : 45000, // en FCFA, entier
  "typePaiement": "SCOLARITE", // SCOLARITE|CANTINE|TRANSPORT|APE|AUTRE
  "trimestre" : "T1", // T1|T2|T3 (stocké dans commentaire)
  "idMode" : 1, // Mode de paiement (1=Especes, 2=MobileMoney...)
  "operation_ID": "MTN-001", // reference transaction (optionnel)
  "datePaie" : "2024-11-18"
}
// Node.js construit : commentaire = SCOLARITE-T1
```

Réponse (JSON) :

```
{
  "idPaie" : 89,
  "matricule" : 2024001,
  "montant" : 45000,
  "typePaiement": "SCOLARITE",
  "trimestre" : "T1",
  "mode" : { "idMode": 1, "libelle": "Especes" },
  "datePaie" : "2024-11-18",
  "numeroRecu" : "RECU-2024-0089",
  "message" : "Paiement enregistré"
}
```

Code	Signification
201	Enregistré
404	Élève introuvable
400	Montant invalide
409	Paiement déjà enregistré ce trimestre

12. MODULE DISCIPLINE

Tables BD : [Rapport](#) | [Discipline](#) | [Justificatifs](#)

La table Rapport stocke les incidents disciplinaires d'un élève (matricule + points + commentaire). La table Discipline liste les types de sanctions avec leur barème de points. La table Justificatifs stocke les pièces justificatives liées à un Rapport.

Méthode	Endpoint	Rôles	Description	Tables BD
GET	/discipline	ADMIN, DIRECTEUR	Lister tous les incidents	Rapport , Discipline
POST	/discipline	ADMIN, DIRECTEUR, ENSEIGNANT	Signaler un incident	Rapport
GET	/discipline/:id	ADMIN, DIRECTEUR	Détail d'un incident	Rapport , Justificatifs
PUT	/discipline/:id	DIRECTEUR	Sanctionner / mettre à jour	Rapport
POST	/discipline/:id/justif	DIRECTEUR	Ajouter pièce justificative	Justificatifs
GET	/discipline/types	Tous (JWT)	Types de sanctions disponibles	Discipline
GET	/eleves/:matricule/discipline	ADMIN, DIRECTEUR	Historique disciplinaire élève	Rapport

POST /discipline — Signaler un incident disciplinaire

Rôles : [\[ADMIN\]](#) [\[DIRECTEUR\]](#) [\[ENSEIGNANT\]](#)

Tables MySQL : [Rapport](#) [Discipline](#)

Corps de la requête (JSON) :

```
// Table Rapport : idRap, libelle, points, matricule, idAca, commentaire, event_date, idPers
{
  "matricule" : 2024003,
  "idAca" : 1,
  "libelle" : "Bavardage en classe",
  "points" : 5, // points de penalite (ref table Discipline)
  "commentaire": "Avertissement oral precedemment donne sans effet",
  "event_date" : "2024-11-18"
}
```

Réponse (JSON) :

```
{
  "idRap" : 12,
  "matricule": 2024003,
  "libelle" : "Bavardage en classe",
  "points" : 5,
  "message" : "Incident enregistre"
}
```

Code	Signification
201	Enregistré
404	Élève introuvable

13. MODULE COMMUNICATION

Tables BD : Messages | Parents | Personne

La table Messages gère les communications entre l'école et les parents. type_message : 0=individuel, 1=tous les parents, 2=parents avec impayés. idExp_Pers = expéditeur (Personne), idParent = destinataire (Parents).

Méthode	Endpoint	Rôles	Description	Tables BD
GET	/messages	ADMIN, DIRECTEUR, SECRETAIRE	Messages envoyés	Messages / Parents
POST	/messages	ADMIN, DIRECTEUR, SECRETAIRE	Envoyer un message	Messages
GET	/messages/:id	ADMIN, DIRECTEUR, SECRETAIRE	Détail d'un message	Messages
GET	/messages/mes-messages	Tous (JWT)	Messages reçus (connecté)	Messages
POST	/messages/diffusion	ADMIN, DIRECTEUR	Message circulaire (type 1/2)	Messages
PUT	/messages/:id/valider	DIRECTEUR	Valider un message	Messages

POST /messages — Envoyer un message à un parent

Rôles : [ADMIN] [DIRECTEUR] [SECRETAIRE]

Tables MySQL : Messages Parents

Corps de la requête (JSON) :

```
// Table Messages : idMessages, idExp_Pers, idParent, objet, information,  
// type_message, AnneeAcade, created_at, valider  
{  
  "idParent" : 1, // ID dans table Parents  
  "objet" : "Convocation – comportement de votre enfant",  
  "information" : "Vous etes convoques le 25/11/2024 a 9h au bureau du directeur.",  
  "type_message": 0, // 0=individuel, 1=tous parents, 2=impayes  
  "AnneeAcade" : "2024-2025"  
}
```

Réponse (JSON) :

```
{  
  "idMessages": 5,  
  "statut" : "ENVOYE",  
  "message" : "Message envoye"  
}
```

Code	Signification
201	Envoyé
404	Parent introuvable

14. MODULE RAPPORTS & EXPORTS (PDF dynamiques)

Tables BD : [Evaluation](#) | [Frequente](#) | [Paielement](#) | [Eleve](#) | [Cours](#) | [Rapport](#)

Tous les rapports sont générés dynamiquement par Node.js (pdfkit) sans table dédiée en base. Les données sont agrégées depuis les tables existantes. Content-Type : application/pdf — réponse binaire.

Méthode	Endpoint	Rôles	Description	Tables BD
GET	/rapports/effectifs	ADMIN, DIRECTEUR	Rapport des effectifs par classe/section	Eleve, Classe, Cycle
GET	/rapports/resultats	ADMIN, DIRECTEUR	Résultats trimestriels (moyennes/classement)	Evaluation, Cours, Eleve
GET	/rapports/presences	ADMIN, DIRECTEUR	Rapport de présences (taux d'assiduité)	Frequent, Eleve, Salle
GET	/rapports/finances	ADMIN, DIRECTEUR	Rapport financier (encaissements/impayés)	Paielement, Eleve, Scolarite
GET	/rapports/discipline	ADMIN, DIRECTEUR	Rapport disciplinaire	Rapport, Discipline, Eleve
GET	/bulletins/eleve/:mat/pdf	ADMIN, DIRECTEUR, PARENT*	Bulletin PDF officiel	Evaluation, Cours, Eleve

Paramètres communs : ?format=pdf|json | ?idTrimes=1 | ?idAca=1 | ?idClasse=1

15. CODES HTTP STANDARDISÉS

Code	Nom HTTP	Quand l'utiliser	Exemple message
200	OK	GET, PUT, PATCH réussis	Données retournées
201	Created	POST — ressource créée	Eleve inscrit avec succes
204	No Content	DELETE réussi	(corps vide)
400	Bad Request	Données invalides / champ manquant	Le champ nom est requis
401	Unauthorized	Token absent, expiré ou invalide	Token JWT expire
403	Forbidden	Rôle insuffisant	Role ENSEIGNANT insuffisant
404	Not Found	Ressource inexistante	Eleve 9999 introuvable
409	Conflict	Doublon / contrainte unicité	Appel deja saisi
423	Locked	Compte inactif (actif=0)	Compte desactive
500	Server Error	Erreur Node.js inattendue	Erreur serveur interne

16. CORRESPONDANCE COMPLÈTE MPD → API

Tableau récapitulatif : pour chaque table du MPD de l'enseignant, les endpoints API correspondants et le module Node.js concerné.

Table MPD	Clé primaire	Endpoints principaux	Module Node.js
Admin	ID	POST /auth/login	authController
AnneeAcademique	idAnnee	GET POST /annees	anneeController
Trimestre	idTrimes	GET POST /trimestres	trimestreController
Session	idSession	GET POST /sessions	sessionController
Cycle	idCycle	GET POST /cycles	cycleController
Classe	idClasse	GET POST PUT DELETE /classes	classeController
Salle	idSalle	GET POST /salles	salleController
Titulaire	idTitulaire	POST /salles/:id/titulaire	titulaireController
VilleNaissance	idVille	GET /villes (référentiel)	villeController
Quartier	idQuartier	GET /quartiers (référentiel)	quartierController
Eleve	matricule	GET POST PUT DELETE /eleves	eleveController
Parents	idParent	GET POST /eleves/:mat/parents	parentController
Residents	idResi	GET /eleves/:mat (inclus dans dossier)	eleveController
Personne	idPers	POST /auth/login, POST /enseignants	authController
Enseignant	idEnseignant	GET POST PUT DELETE /enseignants	enseignantController
FicheEnseignant	idRap	GET POST /enseignants/:id/fiches	ficheController
Cours	idCours	GET POST PUT /cours	coursController
Specialite	idSpecialite	GET POST /specialites	specialiteController
Livres	idLivre	GET POST /livres	livreController
NatureEpreuve	idNature	GET POST /nature-epreuves	natureController
EmploiDuTemps	idTemps	GET POST PUT DELETE /emploiutemps	edtController
Frequente	idFrequente	POST /presences/appe, GET /presences	presenceController
Epreuve	idEpreuve	GET POST /epreuves	epreuveController
Evaluation	idEval	POST /epreuves/:id/notes, GET /evaluations	evaluationController
Scolarite	idScolarite	GET POST /scolarites	scolariteController
Tranches	idTranche	GET POST /scolarites/:id/tranches	trancheController
Mode	idMode	GET /modes	modeController
Paie	idPaie	GET POST PUT DELETE /paiements	paiementController
Rapport	idRap	GET POST PUT /discipline	disciplineController
Discipline	ID	GET /discipline/types	disciplineController
Justificatifs	ID	POST /discipline/:id/justif	disciplineController
Messages	idMessages	GET POST /messages	messageController